AI-Powered Customer Support Ticket & Sentiment Analyzer

An NLP and Machine Learning Approach to Understanding and Categorizing Customer Feedback

PROJECT REPORT

Technology Stack: Python · Pandas · Scikit-learn · Google Colab

Key Technologies: Natural Language Processing (NLP) · Machine Learning · Sentiment Analysis

Prepared by:

Hawa Doumbe Traore

Academic Year:

2026–2027

Table of Contents:

Table of Contents: ---------- 2
GLOSSARY ---------- 5
LIST OF ABBREVIATIONS ---------- 7
GENERAL INTRODUCTION ---------- 8
1. Background and Context: ---------- 8
2. Problem Statement: ---------- 8
3. Project Objectives: ---------- 9
4. Methodology and Scope: ---------- 9
5. Organization of the Report: ----------10
PART I — THEORETICAL ----------11
1. Introduction ----------12
1.1 Context: ----------12
1.2 Problem Statement: ----------12
1.3 Objectives: ----------13
1.4 Scope of the Project: ----------13
1.5 Structure of the Report: ----------13
2. Artificial Intelligence ----------14
2.1 Definition: ----------14
2.2 Machine Learning: ----------14
2.3 Role of AI in Customer Support: ----------15
3. Natural Language Processing ----------15
3.1 Definition: ----------15
3.2 Text Preprocessing: ----------16
3.3 Text Representation: ----------16
3.4 Role of NLP in the Project: ----------17

4. Sentiment Analysis ----------18
4.1 Definition: ----------18
4.2 Sentiment Classification: ----------18
4.3 Approaches to Sentiment Analysis: ----------19
4.4 Importance in Customer Support: ----------19
5. Customer Support Ticket Classification Using Machine Learning ----------20
5.1 Definition: ----------20
5.2 Training a Classification Model: ----------20
5.3 Common Classification Algorithms: ----------21
5.4 Evaluating Model Performance: ----------21
5.5 Distinguishing Ticket Classification from Sentiment Analysis: ----------22
6. Integrating Classification with Sentiment Analysis ----------23
6.1 System Overview: ----------23
6.2 Processing Workflow: ----------23
6.3 Potential Applications: ----------24
7. Technologies and Tools ----------24
7.1 Python: ----------24
7.2 Google Colab: ----------25
7.3 Data Processing Libraries: ----------25
7.4 Machine Learning Libraries: ----------25
7.5 NLP and Visualization Tools: ----------25
8. Limitations and Considerations ----------26
9. Conclusion ----------27
PART II — PRACTICAL ----------28
1. Introduction----------29
2. Development Environment and Overall Approach ----------29

3. Dataset Creation and Exploration ---------- 30
4. Text Preprocessing ---------- 31
5. Training the Ticket Category Classification Model ---------- 33
6. Sentiment Analysis ---------- 37
7. Rule-Based Priority Assignment ---------- 41
8. Integration of the Complete Pipeline ---------- 43
9. Interactive Interface in Google Colab ---------- 45
10. Functional Tests, Limitations, and Possible Improvements ---------- 46
11. Conclusion ---------- 48
List of Figures ---------- 49
List of Tables ---------- 50
References ---------- 51
General conclusion ---------- 53

GLOSSARY

Artificial Intelligence (AI): A field of computer science concerned with developing systems capable of performing tasks that typically require human intelligence, such as learning, reasoning, analyzing information, and making decisions.

Accuracy: A performance metric that measures the proportion of correctly classified observations out of the total number of observations evaluated. It is commonly expressed as a percentage.

Balanced Dataset: A dataset in which the different classes contain equal or approximately equal numbers of observations. In this project, the dataset contains 210 customer support tickets distributed equally across seven categories, with 30 tickets per category.

Classification: A machine learning task in which a model assigns an input observation to one or more predefined categories based on learned patterns.

Customer Feedback: Information, opinions, comments, or complaints provided by customers about a product, service, or experience. It can be analyzed to understand customer needs and satisfaction.

Customer Support: A service provided by an organization to assist customers with questions, technical issues, complaints, and other requests.

Customer Support Ticket: A written record of a customer's question, complaint, request, or reported issue submitted to a support service. It contains textual information that can be analyzed and classified.

Dataset: A structured collection of data used for analysis, model training, and testing. In this project, the dataset consists of 210 customer support tickets.

Data Preprocessing: The process of preparing raw data before using it in a machine learning model. For text-based applications, it may include cleaning text, removing irrelevant elements, and preparing textual information for analysis.

Feature: An individual characteristic or variable used by a machine learning model to identify patterns and generate predictions.

Human-in-the-Loop: An approach in which human judgement and intervention are integrated into an automated system. In this project, the model assists with ticket

classification and sentiment analysis, while a human can review and validate the results when necessary.

Machine Learning (ML): A branch of artificial intelligence that enables computer systems to learn patterns from data and use them to make predictions or classifications without being explicitly programmed for every situation.

Model: A mathematical or computational system trained on data to recognise patterns and produce predictions for new observations.

Natural Language Processing (NLP): A field of artificial intelligence that enables computers to process, analyze, and interpret human language in textual or spoken form.

Prediction: The output generated by a trained machine learning model when it processes new input data. In this project, predictions include the ticket category and its sentiment classification.

Sentiment Analysis: A Natural Language Processing task that identifies the emotional tone or opinion expressed in a text. In this project, customer support tickets are classified into three sentiment classes: Positive, Neutral, and Negative.

Text Classification: The process of automatically assigning written text to predefined categories according to its content. In this project, it is used to organize customer support tickets into seven categories.

Text Preprocessing: The preparation of textual data before it is analyzed by an NLP model. It may involve cleaning, normalizing, and transforming text into a suitable representation for machine learning.

Training Dataset: The portion of a dataset used to train a machine learning model so that it can learn patterns and relationships from the available examples.

Testing Dataset: The portion of a dataset reserved for evaluating a trained model on observations that were not used during training. In this project, 168 tickets were allocated to training and 42 to testing.

Training and Testing Split: The division of a dataset into separate subsets for model training and evaluation. This project uses an 80% training and 20% testing split.

Ticket Categorisation: The process of assigning a customer support ticket to a predefined category according to the issue or request described by the customer.

User Interface: The part of an application through which users interact with a system. In this project, the interface allows a user to enter a customer support ticket, initiate its analysis, and view the results.

Version: A particular iteration of software implementation or model. In this project, the sentiment analysis system was improved from Version 1 to Version 2, with the reported test accuracy increasing from 61.90% to 88.10%.

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
CSV	Comma-Separated Values
ML	Machine Learning
NLP	Natural Language Processing
UI	User Interface
V1	Version 1
V2	Version 2

GENERAL INTRODUCTION

1. Background and Context:

In today's digital environment, customer support plays an essential role in maintaining customer satisfaction and building strong relationships between organizations and their customers. Every day, companies receive numerous messages containing questions, complaints, requests, and feedback. Managing this growing volume of customer interactions can be challenging, particularly when support teams must manually read, organize, and respond to each request.

Artificial Intelligence (AI) and Natural Language Processing (NLP) provide opportunities to improve the way customer communications are handled. These technologies enable computer systems to analyze written language, identify relevant information, categorize messages, and recognize the emotional tone expressed by customers.

Among these applications, text classification and sentiment analysis can help organizations better understand customer requests and identify their priorities. By combining these techniques, an automated system can support customer service teams in organizing incoming tickets and gaining a clearer understanding of customer opinions and concerns.

2. Problem Statement:

When customer support requests are managed entirely manually, several challenges may arise, including time-consuming ticket organization, inconsistent categorization, and difficulties in identifying negative customer experiences.

Furthermore, understanding the actual content of a customer message is not always sufficient. Recognizing its sentiment is also important, as a complaint expressing frustration may require different attention from a neutral information request.

This project therefore addresses the following question: How can Natural Language Processing and Machine Learning be used to automatically categorize customer support tickets and identify the sentiment expressed in each message, while maintaining human supervision over the final decisions?

3. Project Objectives:

The main objective of this project is to develop an AI-based customer support ticket analysis system that combines text classification and sentiment analysis to facilitate the organization and interpretation of customer messages.

The specific objectives are to:

- Develop a system capable of automatically assigning customer support tickets to predefined categories.
- Analyze the textual content of customer messages to identify their sentiment as Positive, Neutral, or Negative.
- Apply machine learning techniques to improve the classification process.
- Evaluate the performance of the developed models using training and testing datasets.
- Create a simple user interface that allows users to enter a ticket and obtain its predicted category and sentiment.
- Integrate a human-in-the-loop approach in which human judgement remains available to review and validate automated predictions.

4. Methodology and Scope:

The project was implemented using Python in Google Colab, with libraries and tools for data processing, machine learning, and application development.

The prepared dataset contains 210 customer support tickets distributed across seven categories, with 30 tickets per category. For sentiment

analysis, the messages are classified into three groups: Positive, Neutral, and Negative.

The dataset was divided into training and testing subsets, consisting of 168 and 42 records, respectively. Two classification tasks were developed: customer support ticket categorization and sentiment analysis. The sentiment analysis component was progressively improved through two model versions, with the reported test accuracy increasing from 44.44% in Version 1 to 88.10% in Version 2.

Finally, the developed system was integrated into a simple interactive interface where users can submit a customer support message and obtain its predicted category and sentiment. The system is designed to assist support teams rather than replace human decision-making.

The project is developed as an academic prototype using a limited dataset. Consequently, the obtained results demonstrate the functionality of the proposed approach but do not guarantee equivalent performance on larger, more diverse, or real-world customer support datasets.

5. Organisation of the Report:

This report is organized into two main parts:

The first part presents the theoretical foundations of the project. It introduces Artificial Intelligence, Machine Learning, Natural Language Processing, text classification, sentiment analysis, and the main concepts related to automated customer support ticket processing.

The second part focuses on practical implementation. It describes the development environment, dataset preparation, data processing, model development, training and testing procedures, performance evaluation, and integration of the interactive user interface. It also presents the results obtained and discusses the limitations and possible improvements of the developed system.

PART I — THEORETICAL

1. Introduction

1.1 Context:

Customer support plays an essential role in maintaining the relationship between a company and its customers. When customers encounter problems, request information, or provide feedback, support teams must understand their needs and respond appropriately.

With the growth of digital platforms, companies receive customer requests through various channels, including emails, online forms, live chats, and social media. This increase in communication generates large volumes of textual data that can be difficult to process manually. Support agents may spend considerable time reading, categorizing, and evaluating incoming messages, potentially delaying responses, and making consistent analysis more challenging.

Artificial Intelligence (AI), particularly Natural Language Processing (NLP) and Machine Learning (ML), offers techniques for analyzing textual information automatically. These technologies can help identify the subject of a customer request and determine the sentiment expressed in the message.

1.2 Problem Statement:

The growing volume of customer support tickets creates several challenges for organizations. Manually identifying the subject of each ticket and interpreting the customer's sentiment can be time-consuming, especially when requests contain different levels of complexity or emotional intensity.

Without automated assistance, support teams may find it difficult to organize incoming requests consistently and identify recurring customer concerns efficiently.

This project therefore addresses the following question: **How can Artificial Intelligence and Machine Learning techniques be used to classify**

customer support tickets and analyze the sentiment expressed in customer messages?

1.3 Objectives:

The main objective is to develop a system that analyzes customer support messages using Python and machine-learning techniques.

The specific objectives are to:

- Process and prepare textual data for analysis.
- Classify customer support tickets into relevant categories using the selected classification model.
- Analyze the sentiment expressed in customer messages.
- Present the results in a clear and interpretable format.
- Examine how automated text analysis can support customer service operations.

1.4 Scope of the Project:

The project focuses on text-based customer support ticket classification and sentiment analysis. Its scope depends on the dataset, the models implemented, and the functions demonstrated in the practical part.

The system is intended to support the analysis of customer messages rather than replace human support agents. Its predictions may assist with understanding customer requests, but they should not be considered definitive judgments about a customer's intentions, emotional state, or the urgency of a problem.

1.5 Structure of the Report:

The theoretical part introduces the principal concepts underlying the project. It presents Artificial Intelligence, Natural Language Processing, sentiment analysis, and machine-learning-based text classification. It then explains how these components can be integrated into a customer support

analysis system before discussing the technologies used and the main limitations.

2. Artificial Intelligence

2.1 Definition:

Artificial Intelligence is a field of computer science concerned with developing systems capable of performing tasks that normally require human cognitive abilities, such as learning, reasoning, recognizing patterns, and interpreting information.

AI includes several subfields, including Machine Learning, Deep Learning, and Natural Language Processing. These approaches enable computer systems to extract patterns from data and use them to produce predictions or classifications.

In customer support, AI can be applied to analyze written requests, identify common topics, detect patterns in customer feedback, and assist support teams in processing large volumes of information.

2.2 Machine Learning:

Machine Learning is a subfield of AI that enables systems to learn patterns from data instead of relying exclusively on explicitly programmed rules.

A machine-learning model is trained using examples that allow it to identify relationships between input information and expected outputs. Once trained, the model can apply the learned patterns to new data.

Machine-learning approaches are commonly divided into three categories:

- **Supervised learning:** The model learns from labeled examples containing both input data and known outputs. Text classification is a common application.

- **Unsupervised learning:** The model identifies structures or patterns in data without predefined target labels, as in clustering.
- **Reinforcement learning:** An agent learns to make decisions through interactions with an environment and feedback in the form of rewards or penalties.

For this project, supervised learning is relevant if the training dataset contains predefined ticket categories or sentiment labels. The exact approach must correspond to the method implemented in the practical part.

2.3 Role of AI in Customer Support:

AI can help organizations analyze customer communications more efficiently by automating repetitive text-processing tasks. For example, a classification model can suggest the category of an incoming ticket, while a sentiment-analysis component can estimate whether its wording expresses positive, negative, or neutral sentiment.

These outputs can help support teams identify recurring issues and understand general patterns in customer feedback. However, their reliability depends on the quality and representativeness of the data, the selected algorithms, and the evaluation process.

AI should therefore be regarded as a decision-support tool whose outputs may require human review.

3. Natural Language Processing

3.1 Definition:

Natural Language Processing is a field of AI that enables computers to process, analyze, and interpret human language. It combines linguistic concepts with computational methods to extract useful information from written or spoken communication.

NLP is particularly relevant to customer support because tickets are primarily composed of natural-language messages. These messages may contain spelling errors, abbreviations, informal expressions, and context-dependent meanings, which can make automated analysis challenging.

3.2 Text Preprocessing:

Before textual data can be analyzed by a machine-learning model, it often requires preprocessing. The purpose is to standardize the text and prepare it for subsequent analysis.

Common preprocessing operations include:

- **Lowercasing:** Converting text to lowercase to reduce differences caused by capitalization.
- **Tokenization:** Dividing a message into smaller units, such as words or tokens.
- **Punctuation handling:** Removing or normalizing punctuation when appropriate for the selected method.
- **Stop-word removal:** Removing frequent words that contribute little to a particular task, when suitable.
- **Text normalization:** Standardizing selected textual forms to improve consistency.

Not every operation is appropriate for every dataset. For example, negation words such as “not” and “never” can be essential for sentiment analysis. Removing them indiscriminately may change the meaning of a message.

Preprocessing must therefore be selected according to the language, dataset, and modeling approach.

3.3 Text Representation:

Machine-learning algorithms generally require numerical inputs. Text representation converts written messages into numerical features that a model can process.

One common technique is **Term Frequency–Inverse Document Frequency (TF-IDF)**. It assigns weights to terms according to their frequency in a document and their distribution across a collection of documents.

Term frequency reflects how often a term appears in a particular document, while inverse document frequency reduces the relative importance of terms that occur across many documents.

TF-IDF can therefore emphasize terms that help distinguish one message from another. For example, words associated with billing, delivery, or technical errors may help a model differentiate between support-ticket categories.

However, TF-IDF primarily represents the statistical importance of terms. It does not fully capture word order, context, sarcasm, or complex semantic relationships.

TF-IDF is a feature-extraction technique, not a classification algorithm. It can be combined with a separate machine-learning model, such as Logistic Regression or Multinomial Naive Bayes, if that combination is used in the implementation.

3.4 Role of NLP in the Project:

NLP provides the processing techniques required to transform customer messages into a representation suitable for automated analysis.

In this project, its role is to prepare textual data and, if implemented, convert messages into numerical features. These representations can then be supplied to the relevant classification models.

The quality of this stage can influence the results of both ticket classification and sentiment analysis because errors or information loss during preprocessing may affect the features available to the models.

4. Sentiment Analysis

4.1 Definition:

Sentiment analysis is an NLP task that seeks to identify the evaluative tone expressed in a text. Depending on the chosen approach, it may classify a message as positive, negative, or neutral.

In customer support, sentiment analysis can help summarize customer feedback and identify messages whose wording indicates dissatisfaction, satisfaction, or a lack of clearly positive or negative sentiment.

However, sentiment labels represent model predictions based on textual evidence. They do not provide a definitive measurement of a customer's emotional state.

4.2 Sentiment Classification:

In a basic sentiment-analysis system, each message is assigned to a sentiment label according to the model's output.

- **Positive sentiment:** The message contains wording associated with satisfaction, appreciation, or a favorable experience.
- **Negative sentiment:** The message contains wording associated with dissatisfaction, complaints, frustration, or an unfavorable experience.
- **Neutral sentiment:** The message contains no clearly positive or negative evaluative language, or the model assigns it to the neutral category.

For example, “Thank you for resolving my issue” may express positive sentiment, while “My payment was deducted twice” may be classified as negative depending on the context and model. A message such as “Please provide the status of my order” may be neutral.

These examples illustrate possible interpretations; actual predictions depend on the implemented model and its training data.

4.3 Approaches to Sentiment Analysis:

Sentiment analysis can be performed using different methods:

- **Lexicon-based approaches** use dictionaries that associate words or expressions with sentiment values. They can be relatively straightforward to implement but may struggle with context, negation, sarcasm, and domain-specific language.
- **Machine-learning approaches** learn patterns from labeled examples. They can identify combinations of terms associated with particular sentiment categories, provided that suitable training data are available.
- **Deep-learning and transformer-based approaches** can model more complex contextual relationships in language. Their performance depends on the architecture, training data, and task-specific adaptation, and they may require more computational resources.

The method described in the final report must be the one actually used in the project. These approaches should not be presented as implemented features if they are discussed only for theoretical comparison.

4.4 Importance in Customer Support:

Sentiment analysis can provide an additional perspective on customer messages. For example, a company may examine the proportion of negative messages across different ticket categories to identify recurring sources of dissatisfaction.

Nevertheless, sentiment and urgency are different concepts. A negative message is not necessarily urgent, and a neutral message can describe a critical technical or financial problem. Similarly, a strongly worded message does not automatically indicate that a request has high operational priority.

Sentiment analysis should therefore complement, rather than replace, an explicit urgency-classification or prioritization mechanism.

5. Customer Support Ticket Classification Using Machine Learning

5.1 Definition:

Text classification is an NLP task in which a model assigns a predefined category to a text based on its content.

In customer support, ticket classification can help identify the main subject of an incoming request. Depending on the dataset, categories might include billing, technical support, account access, product information, or delivery issues.

The categories used in the project must correspond to the actual labels available in the dataset.

5.2 Training a Classification Model:

In supervised text classification, the model learns from labeled examples. Each training example contains a text message and its corresponding category.

The general workflow includes the following stages:

1. **Data collection:** Obtain a dataset containing customer support messages and the labels required for the task.
2. **Data preparation:** Inspect missing values, duplicates, label consistency, and text quality.
3. **Text preprocessing:** Apply to the selected normalization and tokenization procedures.
4. **Feature extraction:** Convert the text into numerical features using an appropriate representation method, such as TF-IDF.
5. **Model training:** Fit the selected classification algorithm using the training data.
6. **Model evaluation:** Assess performance on data that were not used to train the model.

7. **Prediction:** Apply the trained model to new messages and examine the resulting classifications.

This workflow separates the preparation of input data, the learning process, and the evaluation of predictions.

5.3 Common Classification Algorithms:

Several machine-learning algorithms can be used for text classification:

- **Multinomial Naive Bayes** is a probabilistic algorithm commonly applied to discrete text features, including word counts and suitable non-negative term-weight representations. It is computationally efficient and often serves as a baseline for text-classification tasks.

- **Logistic Regression** estimates class probabilities from input features and is widely used for text classification. Despite its name, it can perform classification rather than only regression.

- **Support Vector Machines (SVMs)** identify decision boundaries that separate classes in a feature space. Linear SVMs are frequently used with sparse text representations such as TF-IDF.

These algorithms have different assumptions and computational characteristics. Their performance depends on the dataset, feature representation, parameter settings, and evaluation method. They should be included in the implementation section only if they were actually tested or used.

5.4 Evaluating Model Performance:

A classification model should be evaluated using data separate from its training data. This helps estimate how well it may perform on previously unseen messages.

Common evaluation metrics include:

- **Accuracy:** The proportion of all predictions that are correct.
- **Precision:** The proportion of predictions assigned to a class that actually belongs to that class.
- **Recall:** The proportion of actual examples of a class that the model correctly identifies.
- **F1-score:** The harmonic mean of precision and recall, balancing both measures.

Accuracy alone may be misleading when some categories occur much more frequently than others. Precision, recall, and F1-score provide additional insight into the model's performance for individual classes.

A confusion matrix provides a more detailed view by comparing actual labels with predicted labels. For a binary classification task, it can distinguish between true positives, true negatives, false positives, and false negatives. For multiclass classification, it shows how examples from each actual category are distributed across predicted categories.

The interpretation of a confusion matrix must state which axis represents actual labels, and which represents predicted labels. Evaluation results should be reported using the actual outputs from the project rather than estimated or invented values.

5.5 Distinguishing Ticket Classification from Sentiment Analysis:

Ticket classification and sentiment analysis are related but separate tasks.

Ticket classification identifies **what the customer is contacting the company about**, while sentiment analysis estimates **the sentiment expressed in the message**.

For example, a message about a failed payment may be classified under “Billing,” while its sentiment may be classified as “Negative.” A message about the same billing issue could express neutral sentiment if it simply requests information.

The two outputs can be considered together to provide a more informative overview of a support request. However, they should remain in distinct predictions unless the implementation explicitly combines them into a single model or decision process.

6. Integrating Classification with Sentiment Analysis

6.1 System Overview:

The proposed system combines text-processing techniques with machine-learning-based analysis to extract two types of information from customer support messages: their likely category and their estimated sentiment.

The general process begins with a customer message. The text is prepared according to the implemented preprocessing pipeline and converted into the representation required by the relevant model. The classification component then predicts the ticket category, while the sentiment component generates a sentiment label or score.

The results can subsequently be presented in a structured format for inspection or further analysis.

6.2 Processing Workflow:

The conceptual workflow can be summarized as follows:

Customer message → Text preprocessing → Feature representation → Classification and sentiment analysis → Results presentation

This diagram represents the overall concept. The practical implementation may contain separate pipelines for ticket classification and sentiment analysis, particularly if the two tasks use different datasets, preprocessing methods, or models.

The architecture should be adjusted to reflect the actual notebook. For example, if sentiment analysis uses a pretrained tool while ticket classification uses a TF-IDF model, the diagram should show these separate components rather than implying that one classifier performs both tasks.

6.3 Potential Applications:

A system of this type may support several customer-service activities:

- Organizing incoming tickets by subject.
- Summarizing sentiment patterns across customer messages.
- Identifying categories associated with recurring negative feedback.
- Supporting the analysis of common customer concerns.
- Providing structured information that human agents can use when reviewing requests.

These are potential applications rather than guaranteed outcomes. Automatic assignment, escalation, response generation, and priority ranking should only be described as implemented capabilities if they have been developed and tested in the project.

7. Technologies and Tools

7.1 Python:

Python is a programming language widely used in data analysis, machine learning, and NLP. Its libraries provide tools for data manipulation, feature extraction, model training, evaluation, and visualization.

In this project, Python serves as the main programming language for implementing the text-analysis workflow.

7.2 Google Colab:

Google Colab is a cloud-based notebook environment that allows users to write and execute Python code in a browser. It supports interactive experimentation and can integrate data files and visualization tools.

For this project, Colab provides an environment in which the dataset can be processed, the model can be trained and evaluated, and the results can be documented.

7.3 Data Processing Libraries:

Libraries such as Pandas and NumPy can support data preparation and numerical operations. Pandas is useful for working with tabular datasets, while NumPy provides numerical data structures and mathematical operations.

Their inclusion in the report should reflect their actual use in the notebook.

7.4 Machine-Learning Libraries:

Scikit-learn provides tools for text feature extraction, model training, data splitting, and evaluation. Its relevant components include TF-IDF vectorization, classification algorithms, and evaluation of metrics.

If the project uses scikit-learn, the report should identify the specific vectorizer, classifier, and metrics employed rather than listing every available algorithm.

7.5 NLP and Visualization Tools:

Depending on the implementation, NLP-related tools may be used for tokenization, text normalization, or sentiment estimation. Visualization libraries may also be used to present class distributions, sentiment patterns, or evaluation results.

Only libraries and tools that are actually used should be presented as part of the implemented system. Any additional tools should be identified as possible alternatives rather than existing components.

8. Limitations and Considerations

Although automated text analysis can help organize customer support information, several limitations must be considered.

Data quality: Incomplete, inconsistent, duplicated, or incorrectly labeled messages can reduce model reliability.

Class imbalance: If certain ticket categories appear much more frequently than others, the model may perform well on common categories while failing to identify less frequent ones.

Context and ambiguity: Customer messages may contain sarcasm, negation, abbreviations, spelling mistakes, or expressions whose meaning depends on context. These characteristics can make both classification and sentiment analysis difficult.

Language variation: A model trained in one language or writing style may not perform reliably on other languages, dialects, or informal expressions.

Generalization: Performance on a test dataset does not guarantee equivalent performance on messages received in a real customer-service environment. Data distributions and customer behavior may change over time.

Privacy: Customer support messages may contain personal or confidential information. Data collection, storage, processing, and sharing should follow applicable privacy requirements and organizational policies.

Human oversight: Model predictions can be incorrect or uncertain. Human review remains important, particularly when decisions could significantly affect customers.

These limitations highlight the importance of appropriate data preparation, reliable evaluation, responsible handling of customer information, and cautious interpretation of model outputs.

9. Conclusion

Artificial Intelligence, Natural Language Processing, and Machine Learning provide useful methods for analyzing customer support messages. Through text preprocessing, feature extraction, and classification, a system can identify likely ticket categories and estimate the sentiment expressed in customer communications.

The combination of ticket classification and sentiment analysis can provide complementary information: the first indicates the subject of a request, while the second estimates its evaluative tone. Together, these outputs can support a more structured understanding of customer feedback.

However, the reliability of the system depends on the quality of the data, the selected techniques, and the evaluation process. Sentiment should not be treated as a direct measure of urgency, and automated predictions should not replace human judgment in situations requiring contextual understanding.

Ultimately, this project demonstrates how text-analysis techniques can be applied to a practical customer support problem. Its value should be assessed through the implemented workflow, measured model performance, and a clear understanding of its capabilities and limitations.

PART II — PRACTICAL

1. Introduction

The project was implemented in Google Colab using Python. The system combines two text-classification models with a rule-based business-logic component. The first model predicts the ticket category, while the second estimates the sentiment expressed in the message. A final function combines these outputs and assigns a priority to support the triage of incoming requests. The prototype is intended as a decision-support tool: it does not replace an agent's judgment or the verification of sensitive cases.

The practical objectives are to develop a prototype that can read a customer's message, identify its category, estimate its sentiment, suggest a priority level, and present the analysis in an interactive interface.

2. Development Environment and Overall Approach

Google Colab was selected to run Python code in a notebook environment accessible through a web browser, without requiring extensive local setup. The notebook brings together data preparation, model training, evaluation, manual testing, and interactive interface. The main libraries used are:

- Pandas and NumPy for data manipulation and inspection;
- Re and string for text cleaning;
- Scikit-learn for the train/test split, TF-IDF vectorization, logistic regression, and evaluation metrics;
- Matplotlib and Seaborn for visualizing class distributions and confusion matrices;
- Ipywidgets and IPython.display for the interactive interface within the notebook.

The pipeline follows a sequential processing flow: **raw ticket → text cleaning → TF-IDF numerical representation → category and sentiment predictions → priority assignment → display of the results.**

3. Dataset Creation and Exploration

The project uses a custom dataset of 210 customer support tickets. Each row contains the ticket text, its reference category, and an annotated sentiment label. The dataset includes seven categories, with 30 tickets per category, and three sentiment classes.

```
support_df = pd.DataFrame(
    data,
    columns=["ticket", "category", "sentiment"]
)
print("Dataset shape:", support_df.shape)
print("\nCategory distribution:")
print(support_df["category"].value_counts())
print("\nSentiment distribution:")
print(support_df["sentiment"].value_counts())
print("\nMissing values:")
print(support_df.isnull().sum())
print("\nDuplicate tickets:")
print(support_df["ticket"].duplicated().sum())
display(support_df.head(10))
```

Figure 1: Dataset creation and quality validation (210 tickets)

Item	Observed value
Total number of tickets	210
Main columns	ticket, category, sentiment
Number of categories	7: Billing, Technical, Account, Order, Delivery, Refund, General
Category distribution	30 tickets per category
Sentiment labels	Neutral: 85; Positive: 70; Negative: 55

Data-quality checks	No missing values or duplicate rows detected in the reported check

Table 1: Summary of dataset characteristics and structural statistics

Initial exploration checks the table in dimensions, inspects sample tickets, and verifies the distribution of labels. The balanced category distribution facilitates comparisons between categories; the sentiment distribution is slightly imbalanced.

```
Duplicate tickets:
0
```

	ticket	category	sentiment
0	I was charged twice for the same purchase.	Billing	Negative
1	I don't recognize this charge on my account.	Billing	Negative
2	Why was I charged an extra fee?	Billing	Negative
3	My payment was declined but money was taken fr...	Billing	Negative
4	I was charged the wrong amount for my order.	Billing	Negative
5	There is an unexpected charge on my bill.	Billing	Negative
6	I think I was billed twice for my subscription.	Billing	Negative
7	My invoice shows an amount that I did not expect.	Billing	Negative
8	My card was charged but I did not receive conf...	Billing	Negative
9	The amount on my bill is incorrect.	Billing	Negative

Figure 2: Preview of the dataset structure and billing samples

4. Text Preprocessing

Classification models do not process sentences directly in the same way a human reader does. Before vectorization, a cleaning function normalizes the tickets by converting text to lowercase, removing URLs, digits, and punctuation, and reducing repeated whitespace. The cleaned text is stored in a separate column so that the original and cleaned versions can be compared.

```python
import re
import string

def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+|https\S+", "", text)
    text = text.translate(
        str.maketrans("", "", string.punctuation)
    )
    text = re.sub(r"\s+", " ", text).strip()
    return text

support_df["clean_ticket"] = support_df["ticket"].apply(clean_text)

display(
    support_df[["ticket", "clean_ticket"]].head(10)
)
```

	ticket	clean_ticket
0	I was charged twice for the same purchase.	i was charged twice for the same purchase
1	I don't recognize this charge on my account.	i dont recognize this charge on my account
2	Why was I charged an extra fee?	why was i charged an extra fee
3	My payment was declined but money was taken fr...	my payment was declined but money was taken fr...
4	I was charged the wrong amount for my order.	i was charged the wrong amount for my order
5	There is an unexpected charge on my bill.	there is an unexpected charge on my bill
6	I think I was billed twice for my subscription.	i think i was billed twice for my subscription
7	My invoice shows an amount that I did not expect.	my invoice shows an amount that i did not expect
8	My card was charged but I did not receive conf...	my card was charged but i did not receive conf...
9	The amount on my bill is incorrect.	the amount on my bill is incorrect

Figure 3: Text preprocessing and cleaning pipeline

This step reduces some purely formal variations. By itself, it does not understand the meaning of a sentence: detecting negation, sarcasm, or context remains a limitation. The original ticket is retained for the priority rules so that useful keywords are not lost during text cleaning.

```python
import re
import string
def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+|https\S+", "", text)
    text = text.translate(
        str.maketrans("", "", string.punctuation)
    )
    text = re.sub(r"\s+", " ", text).strip()
    return text

support_df["clean_ticket"] = support_df["ticket"].apply(clean_text)
print("Original vs cleaned text:")
display(
    support_df[["ticket", "clean_ticket"]].head(10)
)
```

```
Original vs cleaned text:
```

	ticket	clean_ticket
0	I was charged twice for the same purchase.	i was charged twice for the same purchase
1	I don't recognize this charge on my account.	i dont recognize this charge on my account
2	I need a refund for a payment I made.	i need a refund for a payment i made
3	Why was I charged an extra fee?	why was i charged an extra fee
4	My payment was declined but money was taken fr...	my payment was declined but money was taken fr...
5	I was charged the wrong amount for my order.	i was charged the wrong amount for my order
6	There is an unexpected charge on my bill.	there is an unexpected charge on my bill
7	I think I was billed twice for my subscription.	i think i was billed twice for my subscription
8	Can you explain this charge on my statement?	can you explain this charge on my statement
9	Why does my invoice show a different amount?	why does my invoice show a different amount

Figure 4: Feature Extraction & TF-IDF Vectorization

5. Training the Ticket Category Classification Model

The first task is to assign each ticket to one of seven categories: Billing, Technical, Account, Order, Delivery, Refund, or General. The dataset is split into training and test sets using an 80% / 20% split, corresponding to 168 training tickets and 42 test tickets. The stratify parameter preserves the class distribution in both sets as far as possible.

The model has two complementary components: TF-IDF converts words and phrases in each ticket into numerical features, and logistic regression learns how those features relate to the labeled categories. Using one- and two-word n-grams enables the model to consider both individual terms and some short expressions.

```
from sklearn.model_selection import train_test_split

X = support_df["clean_ticket"]
y = support_df["category"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 168
Testing samples: 42
```

Figure 5: Data splitting for model training and testing

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

category_model = Pipeline([
    (
        "tfidf",
        TfidfVectorizer(
            ngram_range=(1, 2),
            max_features=5000,
            stop_words="english"
        )
    ),
    (
        "classifier",
        LogisticRegression(max_iter=1000)
    )
])

category_model.fit(X_train, y_train)

print("Category classification model trained successfully.")
```

Figure 6: Category classification model pipeline construction and training

```python
from sklearn.metrics import (
    accuracy_score,
    classification_report
)

y_pred_category = category_model.predict(X_test)

category_accuracy = accuracy_score(
    y_test,
    y_pred_category
)

print(f"Category Model Accuracy: {category_accuracy:.2%}")

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred_category
    )
)
```

```
Category Model Accuracy: 88.10%

Classification Report:
              precision    recall  f1-score   support

     Account       1.00      0.83      0.91         6
     Billing       0.80      0.67      0.73         6
    Delivery       0.83      0.83      0.83         6
     General       1.00      1.00      1.00         6
       Order       0.86      1.00      0.92         6
      Refund       0.86      1.00      0.92         6
   Technical       0.83      0.83      0.83         6

    accuracy                           0.88        42
   macro avg       0.88      0.88      0.88        42
weighted avg       0.88      0.88      0.88        42
```

Figure 7: Category model evaluation and classification performance report

The reported final accuracy is 88.10%. This metric represents the proportion of tickets correctly classified in the test set; it should not be confused with the precision metric for an individual class. The classification report (precision, recall, and F1-score) and confusion matrix provide additional information about which categories are recognized well and which are sometimes confused.

Evaluation	Result	Interpretation
Dataset size	210 tickets	A limited custom dataset
Training / test split	168 / 42	80% / 20% split
Category accuracy	88.10%	Approximately 37 of 42 test tickets correctly classified

Table 2: Summary of model evaluation results, split metrics, and accuracy interpretation

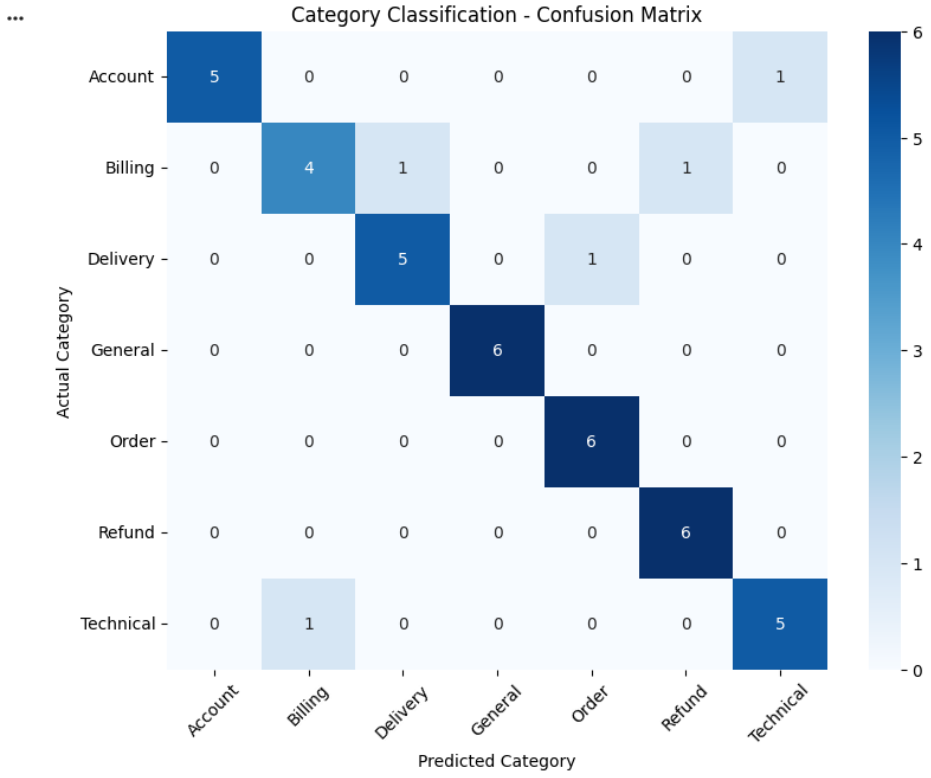


Figure 8: Category classification confusion matrix visualization

```
============================================================
Ticket: I need to cancel my order because I can no longer afford it.
Predicted Category: Order
Confidence: 44.08%
============================================================
Ticket: My package has not arrived yet.
Predicted Category: Delivery
Confidence: 58.81%
============================================================
Ticket: I was charged twice for the same purchase.
Predicted Category: Billing
Confidence: 37.29%
============================================================
Ticket: I want to know how I can get a refund.
Predicted Category: Refund
Confidence: 43.87%
============================================================
Ticket: I forgot my password and cannot access my account.
Predicted Category: Account
Confidence: 55.13%
============================================================
Ticket: The application keeps crashing when I try to open it.
Predicted Category: Technical
Confidence: 41.71%
============================================================
Ticket: What are your opening hours?
Predicted Category: General
Confidence: 32.04%
```

Figure 9: Category prediction inference results and confidence scores on sample tickets

6. Sentiment Analysis

The second model addresses a different task: determining whether a message expresses positive, neutral, or negative sentiment. It also uses TF-IDF and logistic regression, but it learns from the sentiment column. Keeping the models separate makes it possible to evaluate ticket categorization and message tone independently.

```python
from sklearn.model_selection import train_test_split

X_sentiment = support_df["clean_ticket"]
y_sentiment = support_df["sentiment"]

X_train_sentiment, X_test_sentiment, y_train_sentiment, y_test_sentiment = train_test_split(
    X_sentiment,
    y_sentiment,
    test_size=0.20,
    random_state=42,
    stratify=y_sentiment
)

print("Training samples:", len(X_train_sentiment))
print("Testing samples:", len(X_test_sentiment))
```

```
Training samples: 168
Testing samples: 42
```

Figure 10: Dataset split for sentiment analysis training and testing

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

sentiment_model = Pipeline([
    (
        "tfidf",
        TfidfVectorizer(
            ngram_range=(1, 2),
            max_features=5000,
            stop_words="english"
        )
    ),
    (
        "classifier",
        LogisticRegression(max_iter=1000)
    )
])

sentiment_model.fit(
    X_train_sentiment,
    y_train_sentiment
)

print("Sentiment classification model trained successfully.")
```

```
Sentiment classification model trained successfully.
```

Figure 11: Sentiment analysis model pipeline construction and training

```python
from sklearn.metrics import (
    accuracy_score,
    classification_report
)

y_pred_sentiment = sentiment_model.predict(
    X_test_sentiment
)

sentiment_accuracy = accuracy_score(
    y_test_sentiment,
    y_pred_sentiment
)

print(f"Sentiment Model Accuracy: {sentiment_accuracy:.2%}")

print("\nClassification Report:")
print(
    classification_report(
        y_test_sentiment,
        y_pred_sentiment
    )
)
```

```
Sentiment Model Accuracy: 61.90%

Classification Report:
              precision    recall  f1-score   support

    Negative       0.60      0.27      0.38        11
     Neutral       0.57      0.76      0.65        17
    Positive       0.71      0.71      0.71        14

    accuracy                           0.62        42
   macro avg       0.63      0.58      0.58        42
weighted avg       0.62      0.62      0.60        42
```

Figure 12: Sentiment model evaluation and classification performance report

During development, an initial version of the model achieved 61.90% accuracy. After adjustments to the final project version, the reported result increased to 88.10%, a rise of 26.20 percentage points. This improvement illustrates the value of iterating data preparation and model configuration. However, it should be interpreted cautiously because it was measured on a small custom dataset.

Version tested	Reported accuracy	Comment
Initial version	61.90%	Baseline result; not sufficient for autonomous use
Improved version	88.10%	Better result on the test setup used; external validation is still needed
Difference	+26.20 percentage points	Observed improvement between the two versions

Table 3: Comparison of baseline and improved sentiment model accuracy performance

The sentiment confusion matrix shows whether neutral messages are misclassified as positive or negative, or vice versa. This check matters because an incorrect sentiment prediction can influence ticket triage when it is combined with the priority rules.

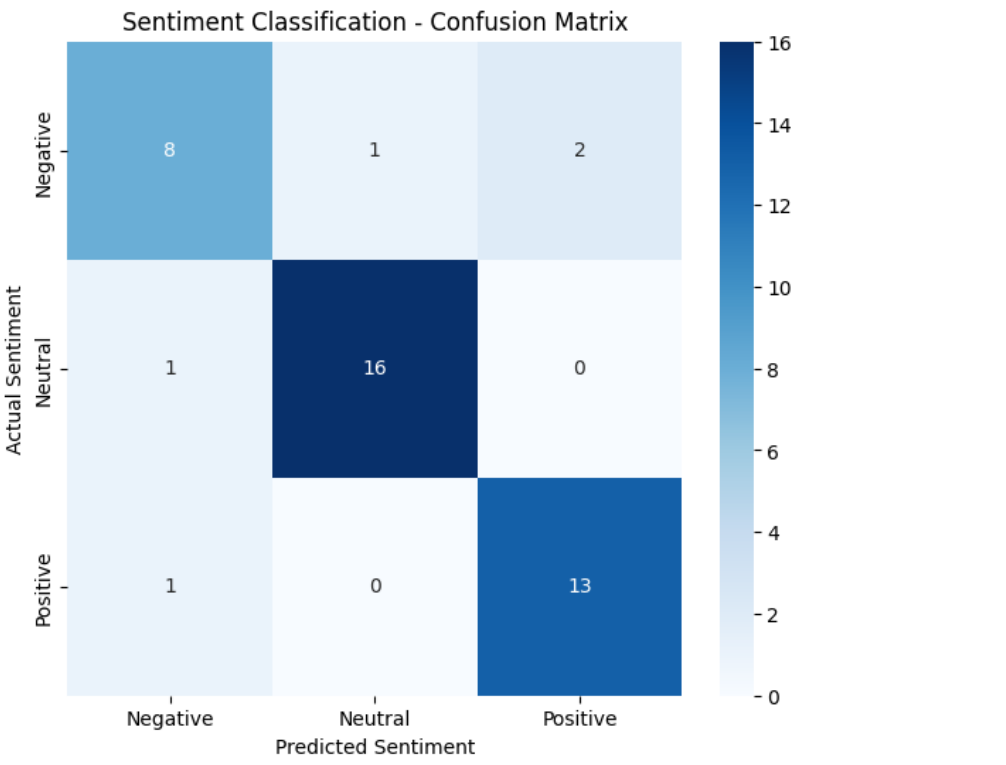


Figure 13: Sentiment classification confusion matrix visualization

```
•••  Sentiment Model V2 Accuracy: 88.10%

     Classification Report:
                   precision    recall  f1-score   support

         Negative       0.80      0.73      0.76        11
          Neutral       0.94      0.94      0.94        17
         Positive       0.87      0.93      0.90        14

         accuracy                           0.88        42
        macro avg       0.87      0.87      0.87        42
     weighted avg       0.88      0.88      0.88        42
```

Figure 14: Sentiment Model V2 evaluation and classification performance report

7. Rule-Based Priority Assignment

Priority is not predicted by a third machine-learning model. Instead, an explicit rule-based function examines keywords in the original ticket and considers the predicted sentiment. This design makes the triage criteria easier to inspect and explain. The four priority levels are Low, Medium, High, and Urgent.

Priority level	General rule	Example situation
Urgent	Keywords related to fraud, unauthorized access, a compromised account, or an explicit emergency	Unauthorized payment
High	Negative sentiment combined with a significant issue, such as a prolonged delay, outage, failure, or strong complaint	Severely delayed order
Medium	Negative sentiment without an indicator that triggers a higher level	Routine account-access difficulty

Low	Positive or neutral message, or an information request without a critical keyword	Thank-you message or general question

Table 4: Summary of priority levels, governing rules, and corresponding customer support scenarios

```python
def determine_priority(ticket, sentiment, category):
    ticket_lower = ticket.lower()
    urgent_keywords = [
        "fraud",
        "hacked",
        "stolen",
        "unauthorized",
        "emergency",
        "immediately",
        "urgent",
        "charged twice"
    ]
    high_keywords = [
        "angry",
        "extremely disappointed",
        "unacceptable",
        "terrible",
        "not working",
        "broken",
        "failed",
        "damaged",
        "late",
        "delayed",
        "two weeks",
        "weeks",
        "nobody has contacted",
        "no response",
        "still waiting"
    ]


    if any(word in ticket_lower for word in urgent_keywords):
        return "Urgent"

    if sentiment == "Negative":

        if any(word in ticket_lower for word in high_keywords):
            return "High"

        if category in [
            "Billing",
            "Delivery",
            "Technical",
            "Refund",
            "Account"
        ]:
            return "High"

    if sentiment in ["Neutral", "Positive"]:
        return "Low"

    return "Medium"
```

Figure 15: Rule-based priority determination function definition and logic implementation

The order of the conditions matters: the most critical rules are applied before the more general rules. In an operational system, the keyword lists should be validated with support agents, extended with test cases, and reviewed regularly. A simple rule can still misinterpret a sentence whose meaning depends on context.

8. Integration of the Complete Pipeline

The analyze_customer_ticket function cleans the ticket, predicts its category and sentiment, obtains the highest predict_proba scores, and assigns priority. These scores are model indicators, not guarantee correctness; reliable probability interpretation would require calibration.

```python
def analyze_customer_ticket(ticket):
    cleaned_ticket = clean_text(ticket)
    category = category_model.predict(
        [cleaned_ticket]
    )[0]

    category_probabilities = category_model.predict_proba(
        [cleaned_ticket]
    )[0]

    category_confidence = category_probabilities.max()
    sentiment = sentiment_model_v2.predict(
        [cleaned_ticket]
    )[0]

    sentiment_probabilities = sentiment_model_v2.predict_proba(
        [cleaned_ticket]
    )[0]

    sentiment_confidence = sentiment_probabilities.max()
    priority = determine_priority(
        ticket,
        sentiment,
        category
    )

    return {
        "Ticket": ticket,
        "Category": category,
        "Category Confidence": f"{category_confidence:.2%}",
        "Sentiment": sentiment,
        "Sentiment Confidence": f"{sentiment_confidence:.2%}",
        "Priority": priority
    }
```

Figure 16: End-to-end customer support ticket analysis and inference pipeline function

Final test: customer reports for account access and an unauthorized payment. The model predicts Account and Negative; the rule engine assigns Urgent.

```
CUSTOMER SUPPORT AI ANALYZER
==================================================
Ticket: Someone accessed my account and made an unauthorized payment.
--------------------------------------------------
Category: Account
Category Confidence: 38.40%
Sentiment: Negative
Sentiment Confidence: 58.21%
Priority: Urgent
```

Figure 17: Customer support AI analyzer sample output and triage recommendation

The **38.40%** category score is low, so this output should support—not determine—triage. The unauthorized payment keyword triggers Urgent, requiring prompt human review.

```
======================================================================
Ticket: I was charged twice for the same purchase and need help immediately.
Category: Billing
Category Confidence: 31.25%
Sentiment: Negative
Sentiment Confidence: 75.45%
Priority: Urgent
======================================================================
Ticket: My package has not arrived and I have been waiting for two weeks.
Category: Delivery
Category Confidence: 58.81%
Sentiment: Negative
Sentiment Confidence: 78.52%
Priority: High
======================================================================
Ticket: Thank you, your support team solved my account problem.
Category: Account
Category Confidence: 23.76%
Sentiment: Positive
Sentiment Confidence: 80.12%
Priority: Low
======================================================================
Ticket: The application keeps crashing and I cannot use the service.
Category: Technical
Category Confidence: 35.20%
Sentiment: Negative
Sentiment Confidence: 42.14%
Priority: High
======================================================================
Ticket: I would like to know how I can request a refund.
Category: Refund
Category Confidence: 40.26%
Sentiment: Neutral
Sentiment Confidence: 89.56%
Priority: Low
======================================================================
Ticket: What are your customer service opening hours?
Category: General
Category Confidence: 49.00%
Sentiment: Neutral
Sentiment Confidence: 72.22%
Priority: Low
```

Figure 18: Batch customer support ticket analysis and multi-priority inference output examples

9. Interactive Interface in Google Colab

A lightweight interface was created with ipywidgets. It includes a text area for entering a new ticket, an analysis button, and an output area. When the button is clicked, the text is passed to the analysis function, and the result is displayed directly in the notebook. This interface makes manual testing easier because the code does not need to be modified for each new request.

```
import ipywidgets as widgets
from IPython.display import display

ticket_input = widgets.Textarea(
    value="",
    placeholder="Enter a customer support ticket...",
    description="Ticket:",
    layout=widgets.Layout(width="700px", height="100px")
)

analyze_button = widgets.Button(
    description="Analyze Ticket",
    button_style="primary"
)

output = widgets.Output()
```

Figure 19: Interactive widgets setup and component definition for the customer support analysis interface

```
def on_analyze_button_clicked(b):

    with output:
        output.clear_output()

        ticket = ticket_input.value.strip()

        if not ticket:
            print("Please enter a customer support ticket.")
            return

        result = analyze_customer_ticket(ticket)

        print("=" * 60)
        print("AI CUSTOMER SUPPORT ANALYZER")
        print("=" * 60)

        print(f"Ticket: {result['Ticket']}")
        print()

        print(f"Category: {result['Category']}")
        print(f"Category Confidence: {result['Category Confidence']}")
        print()

        print(f"Sentiment: {result['Sentiment']}")
        print(f"Sentiment Confidence: {result['Sentiment Confidence']}")
        print()

        print(f"Priority: {result['Priority']}")
        print("=" * 60)


analyze_button.on_click(on_analyze_button_clicked)
```

Figure 20: Interactive button event handler implementation and callback binding for the customer support analysis interface

The interface runs within the Colab session; it is not yet a deployed web application or a service connected to a real support platform. Nevertheless, it demonstrates the end-to-end workflow and makes it possible to test different messages.

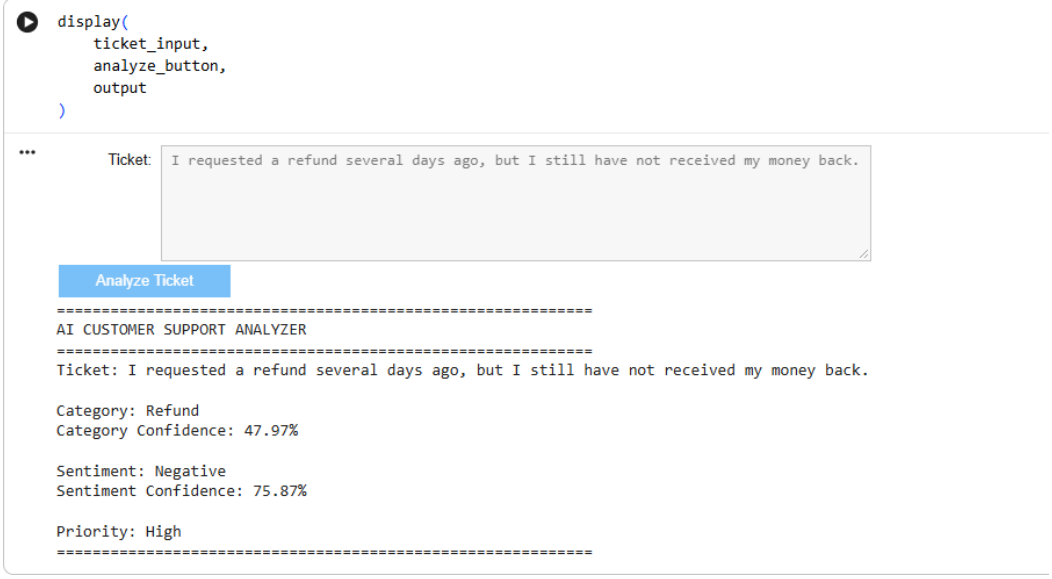


Figure 21: Interactive widget execution and sample ticket analysis interface

10. Functional Tests, Limitations, and Possible Improvements

Manual tests covered technical issues, account access, billing complaints, information requests, and positive messages. They confirmed that the pipeline returned to the expected fields but did not replace evaluation on a larger independent dataset.

The 7/7 result is based on manually selected cases with known expected outputs. It is a functional check, not the 42-ticket test accuracy, and does not establish performance on unseen messages. The main limitations of this experiment are:

- The dataset contains only 210 custom tickets, so it may not fully represent the messages received by a real customer support team.
- Performance depends on the train/test split; one split alone cannot guarantee generalization to new customers and messages.
- The displayed confidence scores are not necessarily calibrated and should be interpreted as model indicators.
- The priority rules rely on keywords: they may miss unexpected phrasings or assign an overly high priority in some contexts.
- The prototype is not deployed in production. Human review should remain part of the process, especially for unauthorized payments, suspected fraud, and account-access problems.

Future work should expand the dataset, validate independent data, inspect per-class precision/recall/F1, calibrate confidence scores, and test priority rules with scenarios reviewed by support agents.

Final result	Reported value
Category accuracy	88.10%
Sentiment accuracy — initial version	61.90%
Sentiment accuracy — improved version	88.10%
Category for the unauthorized-payment example	Account (confidence: 38.40%)
Sentiment for the example	Negative (confidence: 58.21%)
Targeted manual tests	7/7 expected outputs for the cases checked
Priority for the example	Urgent

Table 5: Summary of final results, model accuracies, and evaluation metrics

11.Conclusion

This project was developed as a Python-based natural language processing pipeline designed to automate customer support operations by predicting ticket categories, evaluating user sentiment, and dynamically assigning priority status to incoming requests. By combining TF-IDF feature extraction with logistic regression classifiers, the system achieves a strong balance between computational efficiency, model interpretability, and predictive accuracy. To make the pipeline accessible for real-time testing, an interactive interface was implemented directly in Google Colab, allowing users to input raw ticket text and receive instant classification, sentiment scoring, and priority routing.

During evaluation, both the categorization and sentiment analysis models achieved a matching accuracy of 88.10% on the test split. While these results demonstrate strong initial baseline performance and confirm that linear models with TF-IDF vectorization can effectively capture semantic context in customer inquiries, the current dataset is limited to only 210 tickets. Consequently, independent validation on a larger, more diverse dataset is required to evaluate the models' true generalization capabilities, mitigate the risk of overfitting, and ensure robust performance across varied vocabulary, phrasing, and edge cases.

From an operational perspective, this tool is designed to augment and support human customer support agents rather than replace them. By automating repetitive tasks such as initial ticket sorting, sentiment tracking, and priority tagging, the pipeline significantly reduces manual workload, minimizes response times, and helps teams address urgent issues faster. Looking forward, future enhancements to this project could include expanding the dataset, experimenting with lightweight transformer-based architectures for improved contextual understanding, and integrating the pipeline into an enterprise ticketing workflow via a dedicated REST API or web framework.

List of Figures

Figure 1: Dataset creation and initial data loading pipeline setup 30

Figure 2: Initial dataset preview and sample records inspection 31

Figure 3: Text preprocessing, lowercasing, and punctuation removal implementation 32

Figure 4: Tokenization and stop word removal processing steps 33

Figure 5: TF-IDF feature extraction vectorization configuration 34

Figure 6: Category classification model training setup and pipeline definition 34

Figure 7: Category model performance metrics and classification report 35

Figure 8: Initial sentiment model (Version 1) baseline training and evaluation 36

Figure 9: Category prediction inference results and confidence scores 37

Figure 10: Category classification confusion matrix visualization 38

Figure 11: Category classification model evaluation report 38

Figure 12: Sentiment model evaluation and classification performance report 39

Figure 13: Sentiment classification confusion matrix visualization 40

Figure 14: Sentiment Model V2 evaluation and classification performance report 41

Figure 15: Rule-based priority determination function 42

Figure 16: End-to-end customer support ticket analysis and inference pipeline function 43

Figure 17: Customer support AI analyzer sample output and triage recommendation . 44

Figure 18: Batch customer support ticket analysis and multi-priority inference output examples 44

Figure 19: Interactive widgets setup and component definition for the customer support analysis interface 45

Figure 20: Interactive button event handler implementation and callback binding for the customer support analysis interface 42

Figure 21: Interactive widget execution and sample ticket analysis interface 46

List of Tables

Table 1: Summary of dataset characteristics and structural statistics 30

Table 2: Summary of model evaluation results, split metrics, and accuracy interpretation 36

Table 3: Comparison of baseline and improved sentiment model accuracy performance 40

Table 4: Summary of priority levels, governing rules, and corresponding customer support scenarios 41

Table 5: Summary of final results, model accuracies, and evaluation metrics 47

References

Google. (n.d.). *Google Colaboratory FAQ*. Google Research. https://research.google.com/colaboratory/faq.html

Holzinger, A. (2016). Interactive machine learning for health informatics: When do we need the human-in-the-loop? *Brain Informatics, 3*, 119–131. https://doi.org/10.1007/s40708-016-0042-6

Jurafsky, D., & Martin, J. H. (2026). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with language models* (3rd ed., online manuscript). Stanford University. https://web.stanford.edu/~jurafsky/slp3/

Liu, B. (2012). *Sentiment analysis and opinion mining*. Morgan & Claypool Publishers. https://doi.org/10.2200/S00416ED1V01Y201204HLT016

Liu, B., & Zhang, L. (2012). A survey of opinion mining and sentiment analysis. In C. C. Aggarwal & C. Zhai (Eds.), *Mining text data* (pp. 415–463). Springer. https://doi.org/10.1007/978-1-4614-3223-4_13

Matplotlib Development Team. (n.d.). *Matplotlib documentation*. https://matplotlib.org/stable/

NumPy Developers. (n.d.). *NumPy documentation*. https://numpy.org/doc/stable/

pandas Development Team. (n.d.). *pandas documentation*. https://pandas.pydata.org/docs/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Python Software Foundation. (n.d.). *Python 3 documentation*. https://docs.python.org/3/

Sebastiani, F. (2002). Machine learning in automated text categorization. *ACM Computing Surveys, 34*(1), 1–47. https://doi.org/10.1145/505282.505283

Googlecolab link:
https://colab.research.google.com/drive/1Fm42H3lxT6jOdUg6PJLUc6GLWNuniksg?usp=sharing

General Conclusion

This project provided an opportunity to explore and apply Artificial Intelligence techniques to improve customer support management through automated ticket classification and sentiment analysis. The main objective was to develop an intelligent system capable of analyzing customer messages, identifying the category of each request, and determining the sentiment expressed within the message. Through the implementation of machine learning techniques, the project demonstrated how Artificial Intelligence can contribute to making customer service operations faster, more structured, and more efficient.

Throughout the development process, several important stages were addressed, including data collection, data preparation, text preprocessing, model training, testing, and performance evaluation. The results obtained highlighted the importance of selecting appropriate algorithms, preparing reliable datasets, and continuously evaluating model performance to improve prediction accuracy. The project also demonstrated that combining ticket classification with sentiment analysis can provide customer support teams with a better understanding of incoming requests, helping them identify customer concerns and prioritize responses according to their needs.

Furthermore, this work emphasized the importance of maintaining a balance between automation and human intervention. Although Artificial Intelligence can significantly reduce repetitive tasks and support decision-making, human judgment remains essential when dealing with complex, sensitive, or exceptional customer situations. The developed solution should therefore be considered a decision-support tool rather than a complete replacement for human agents.

Overall, this project strengthened the understanding of how machine learning and Natural Language Processing can be applied to real-world business challenges. It also highlighted the potential of intelligent automation to improve customer experience, optimize operational efficiency, and support the continuous development of modern customer service systems.